

C++ 并发编程与部分现代特性讲义

长空御风视觉组

2026

Contents

C++ 并发编程讲义索引	3
文件列表	4
知识点依赖关系	4
快速查找	5
std::thread — 线程基础	5
概念	5
基本用法	5
join 与 detach	6
传递参数	6
移动语义	6
获取线程 ID	7
异常处理	7
常见陷阱	7
思考题	7
std::mutex + std::lock_guard — 互斥锁与 RAII	8
概念	8
std::mutex 基本用法	8
std::lock_guard — RAII 自动管理	8
缩小临界区范围	8
用花括号限制锁的作用域	9
mutable mutex	9
常见陷阱	10
思考题	10
std::unique_lock + std::condition_variable — 条件等待与唤醒	10
概念	10
std::unique_lock	10
生产者-消费者示例	11
虚假唤醒 (Spurious Wakeup)	11
定时等待	12
notify_one vs notify_all	12
常见陷阱	12
思考题	13

std::shared_mutex + std::shared_lock — 读写锁	13
概念	13
基本用法	13
锁的兼容性	13
shared_lock vs unique_lock vs lock_guard	13
性能对比	14
mutable shared_mutex	14
升级锁（注意事项）	15
常见陷阱	15
思考题	15
std::atomic — 无锁原子操作	15
概念	15
基本用法	15
常用操作	16
exchange — 原子读-改-写	16
compare_exchange — CAS（比较并交换）	16
atomic vs mutex 对比	17
内存顺序（简介）	17
常见陷阱	17
思考题	18
std::chrono — 时间处理	18
概念	18
时间字面量	18
时间点与时钟	18
时间转换	19
定时等待 vs 可中断等待	19
TTL（存活时间）过期机制	20
常见陷阱	20
思考题	20
std::variant + std::visit — 类型安全的多态值	21
概念	21
基本用法	21
取值方式	21
std::visit — 对所有可能类型执行操作	22
与继承体系的对比	22
常见陷阱	23
思考题	23
std::any — 类型擦除的通用容器	23
概念	23
基本用法	23
取值方式	24
实际应用：事件系统	24
模板包装：类型安全的 set/get	25
variant vs any 对比	25

常见陷阱	26
思考题	26
std::future + std::promise + std::packaged_task — 异步结果传递	26
概念	26
std::promise + std::future	26
std::packaged_task	27
为什么 packaged_task 不能放入 std::function?	27
std::async — 最简便的异步方式	28
future 的状态	28
std::shared_future — 多线程共享结果	28
常见陷阱	29
思考题	29
模板、完美转发与参数包展开	29
一、函数模板基础	29
二、可变参数模板 (Variadic Templates)	29
三、包展开语法	30
四、万能引用与完美转发	31
五、lambda 中的参数包捕获 (C++20)	32
六、std::invoke — 统一调用接口	33
七、综合示例：通用日志函数	34
常见陷阱	34
思考题	34
哈希分片 (Sharding) — 降低锁竞争	34
概念	34
核心思路	35
哈希路由	35
完整实现示例	35
分片数量的选择	36
跨分片操作与死锁	36
常见陷阱	37
思考题	37
事务与死锁避免	37
一、死锁	37
二、事务	38
常见陷阱	40
思考题	40

C++ 并发编程讲义索引

本目录包含 C++ 并发编程与现代 C++ 特性的系统讲义，每个文件对应一个知识点，包含概念说明、代码示例、常见陷阱和思考题。

文件列表

同步原语 (01-06)

文件	内容
01_thread_基础.md	<code>std::thread</code> — 创建、join/detach、移动语义、异常处理
02_mutex_与_lock_guard.md	<code>std::mutex</code> + <code>std::lock_guard</code> — 互斥锁、RAII、mutable
03_unique_lock_与_condition_variable.md	<code>std::unique_lock</code> + <code>std::condition_variable</code> — 条件等待、虚假唤醒
04_shared_mutex_读写锁.md	<code>std::shared_mutex</code> + <code>std::shared_lock</code> — 读写锁、并发读
05_atomic_原子操作.md	<code>std::atomic</code> — 无锁操作、exchange、CAS
06_chrono_时间处理.md	<code>std::chrono</code> — 时间字面量、TTL、可中断等待

类型系统 (07-08)

文件	内容
07_variant_与_visit.md	<code>std::variant</code> + <code>std::visit</code> — 类型安全的多态值
08_any_类型擦除.md	<code>std::any</code> — 类型擦除容器，与 <code>variant</code> 对比

异步与模板 (09-10)

文件	内容
09_future_promise_异步.md	<code>std::future</code> + <code>std::promise</code> + <code>std::packaged_task</code>
10_模板_完美转发_参数包.md	可变参数模板、包展开、折叠表达式、完美转发、 <code>std::invoke</code>

架构模式 (11-12)

文件	内容
11_哈希分片.md	Sharding — 降低锁竞争、跨分片死锁避免
12_事务与死锁避免.md	死锁条件、按序加锁、 <code>std::lock</code> 、ACID、write-set

知识点依赖关系

- 01 thread
 - └ 02 mutex + lock_guard
 - └ 03 unique_lock + condition_variable
 - └ 04 shared_mutex (读写锁, 与03对比)
 - └ 05 atomic (轻量级替代, 与mutex对比)

└ 06 chrono (定时等待, 配合cv使用)

07 variant (类型系统)

└ 08 any (与variant对比)

09 future/promise (异步)

└ 10 模板/完美转发 (实现异步接口的基础设施)

11 sharding (架构模式, 基于02-04)

└ 12 事务 (基于02-03, 跨分片操作)

快速查找

症状	对应文件
程序崩溃 (terminate)	01 (忘记 join/detach)
数据竞争 / 结果不确定	02
线程一直等待 / 死锁	03 或 12
读操作互相阻塞	04 (用了独占锁而非共享锁)
计数器不准确	05
后台服务无法立即停止	06 (用了 sleep_for 而非 wait_for)
variant 取值抛异常	07
any_cast 抛异常	08
无法获取任务返回值	09
成员函数指针调用失败	10
高并发写入性能差	11
并发操作数据不一致	12

std::thread — 线程基础

概念

std::thread 代表一个独立执行的线程。创建对象时, 操作系统立即启动新线程并执行传入的函数。

基本用法

```
#include <thread>
#include <iostream>

std::thread t([] {
    std::cout << " 子线程运行中\n";
});
t.join(); // 等待线程结束
```

join 与 detach

```
std::thread t(some_function);

t.join();    // 阻塞当前线程，等待 t 执行完毕
t.detach();  // 放弃管理，线程在后台独立运行
```

规则：析构前必须调用 join() 或 detach() 其中之一，否则析构函数调用 std::terminate() 终止程序。

```
// 错误：忘记 join/detach
{
    std::thread t([] { /* ... */ });
} // 析构时 t 仍是 joinable → 程序崩溃！

// 正确：RAII 包装
{
    std::thread t([] { /* ... */ });
    t.join();
}
```

传递参数

```
void process(const std::string& name, int count) {
    for (int i = 0; i < count; ++i)
        std::cout << name << ": step " << i << "\n";
}

std::thread t(process, "worker", 5);
t.join();
```

参数默认按值拷贝传入线程。传引用必须用 std::ref()：

```
int result = 0;
std::thread t([](int& r) { r = 42; }, std::ref(result));
t.join();
// result == 42
```

移动语义

std::thread 不可拷贝，只能移动：

```
std::thread t1([] { /* ... */ });

// std::thread t2 = t1; // 编译错误！

std::thread t2 = std::move(t1); // t1 失效，t2 接管
t2.join();
```

获取线程 ID

```
std::thread t([] {
    std::cout << " 自身 ID: " << std::this_thread::get_id() << "\n";
});
std::cout << " 外部看到的 ID: " << t.get_id() << "\n";
t.join();
```

异常处理

线程内抛出的异常不会传播到创建线程，会直接 `terminate()`。用 `promise` 传递异常：

```
std::promise<void> p;
auto future = p.get_future();

std::thread t([&p] {
    try {
        // ... 可能抛异常的代码 ...
        p.set_value();
    } catch (...) {
        p.set_exception(std::current_exception());
    }
});
t.join();

try {
    future.get(); // 重新抛出线程内的异常
} catch (const std::exception& e) {
    std::cout << " 捕获到: " << e.what() << "\n";
}
```

常见陷阱

问题	原因	解决方案
程序崩溃 (<code>terminate</code>)	析构前未 <code>join/detach</code>	RAII 包装，析构时自动 <code>join</code>
数据竞争	多线程同时读写同一变量	用 <code>mutex</code> 保护共享数据
悬空引用	线程引用了已销毁的局部变量	确保被引用对象生命周期长于线程
重复 <code>join</code>	对同一线程调用两次 <code>join</code>	调用前检查 <code>t.joinable()</code>

思考题

1. 为什么 `std::thread` 不可拷贝？如果允许拷贝会发生什么？
2. `join()` 和 `detach()` 各适合什么场景？
3. 如何实现一个 RAII 包装类，使 `std::thread` 在析构时自动 `join`？

std::mutex + std::lock_guard — 互斥锁与 RAII

概念

多个线程同时读写同一数据会产生**数据竞争**（data race），结果是未定义行为。

```
int counter = 0;

// 两个线程同时执行，++counter 实际是三步：读 → 加 1 → 写
// 两个线程可能同时读到同一值，各自加 1 后写回，导致只加了一次
void bad_increment() {
    ++counter; // 数据竞争!
}
```

std::mutex 保证同一时刻只有一个线程进入临界区。

std::mutex 基本用法

```
#include <mutex>

std::mutex mtx;
int counter = 0;

void safe_increment() {
    mtx.lock(); // 加锁：若已被持有则阻塞等待
    ++counter; // 临界区
    mtx.unlock(); // 解锁
}
```

问题：临界区内抛出异常时，unlock() 永远不会被调用 → 死锁。

std::lock_guard — RAII 自动管理

```
std::mutex mtx;
int counter = 0;

void safe_increment() {
    std::lock_guard<std::mutex> lock(mtx); // 构造时加锁
    ++counter;
} // lock 析构时自动解锁，即使抛出异常也能正确解锁
```

C++17 支持类模板参数推导：

```
std::lock_guard lock(mtx); // 省略模板参数
```

缩小临界区范围

临界区应尽量小，只保护真正需要保护的代码：

```

std::mutex mtx;
std::unordered_map<int, std::string> cache;

// 不好: 耗时的网络请求在锁内
void bad_update(int id) {
    std::lock_guard lock(mtx);
    std::string data = fetch_from_network(id); // 耗时操作在锁内!
    cache[id] = data;
}

// 好: 只在写入时加锁
void good_update(int id) {
    std::string data = fetch_from_network(id); // 锁外执行
    std::lock_guard lock(mtx);                // 只在写入时加锁
    cache[id] = data;
}

```

用花括号限制锁的作用域

```

std::mutex mtx;
std::queue<std::string> log_queue;

void log(const std::string& msg) {
    {
        std::lock_guard lock(mtx);
        log_queue.push(msg);
    } // 锁在这里释放
    cv.notify_one(); // 解锁后再通知, 避免消费者被唤醒后立刻又等锁
}

```

mutable mutex

const 成员函数不能修改成员变量, 但加锁需要修改 mutex 状态。用 mutable 修饰:

```

class SafeCounter {
public:
    int get() const {
        std::lock_guard lock(mutex_); // const 函数中加锁
        return value_;
    }
    void increment() {
        std::lock_guard lock(mutex_);
        ++value_;
    }
private:
    mutable std::mutex mutex_; // mutable: 允许在 const 函数中修改
    int value_{0};
}

```

```
};
```

常见陷阱

问题	原因	解决方案
忘记解锁	直接用 <code>lock()/unlock()</code>	改用 <code>lock_guard</code>
死锁（自己等自己）	同一线程对同一 <code>mutex</code> 加锁两次	用 <code>std::recursive_mutex</code> 或重构
死锁（互相等待）	两个线程以不同顺序获取两把锁	按固定顺序加锁，或用 <code>std::lock()</code>
锁粒度太大	临界区包含耗时操作	缩小临界区
保护不完整	部分代码路径未加锁	确保所有访问共享数据的路径都加锁

思考题

1. 为什么 `std::mutex` 不可拷贝？
2. `lock_guard` 和 `unique_lock` 的区别是什么？各适合什么场景？
3. 如果两个函数都需要加同一把锁，且一个函数调用另一个，会发生什么？

`std::unique_lock` + `std::condition_variable` — 条件等待与唤醒

概念

当线程需要等待某个条件成立时，不应该忙等（busy-wait）浪费 CPU：

```
// 错误：忙等，100% CPU 占用
while (queue.empty()) { /* 一直循环检查 */ }
```

`std::condition_variable` 让线程在条件不满足时睡眠，条件满足时被唤醒。

`std::unique_lock`

比 `lock_guard` 更灵活，支持手动 `lock()/unlock()` 和延迟加锁，**条件变量必须配合 `unique_lock` 使用**：

```
std::mutex mtx;

std::unique_lock<std::mutex> ul(mtx); // 构造时加锁
ul.unlock(); // 手动解锁
ul.lock(); // 手动重新加锁
// 析构时若持有锁则自动解锁

// 延迟加锁
std::unique_lock<std::mutex> ul2(mtx, std::defer_lock);
// 此时未加锁
ul2.lock(); // 手动加锁
```

生产者-消费者示例

```
std::mutex mtx;
std::condition_variable cv;
std::queue<std::string> message_queue;
bool closed = false;

// 消费者: 等待直到队列非空或已关闭
void consumer() {
    std::unique_lock<std::mutex> lock(mtx);

    // wait 的工作流程:
    // 1. 检查 predicate, 若为 true 则直接继续
    // 2. 若为 false: 释放锁 → 睡眠 → 被唤醒 → 重新加锁 → 再次检查
    // 3. 重复步骤 2, 直到 predicate 返回 true
    cv.wait(lock, [] { return !message_queue.empty() || closed; });

    if (!message_queue.empty()) {
        auto msg = message_queue.front();
        message_queue.pop();
        // 处理消息...
    }
}

// 生产者: 放入消息后唤醒消费者
void producer(const std::string& msg) {
    {
        std::lock_guard<std::mutex> lock(mtx);
        message_queue.push(msg);
    } // 先解锁, 再通知
    cv.notify_one();
}

// 关闭: 唤醒所有等待的消费者
void close() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        closed = true;
    }
    cv.notify_all();
}
```

虚假唤醒 (Spurious Wakeup)

线程可能在没有 notify 的情况下被唤醒, 这是操作系统层面的现象。**必须**使用带 predicate 的 wait:

```
// 错误：不检查条件，虚假唤醒后会错误地继续执行
cv.wait(lock);

// 正确：带 predicate，虚假唤醒后自动重新检查
cv.wait(lock, [&] { return !queue.empty(); });

// 等价的手写循环（理解原理用）
while (queue.empty()) {
    cv.wait(lock);
}
```

定时等待

```
using namespace std::chrono_literals;

std::unique_lock<std::mutex> lock(mtx);

// wait_for: 等待最多指定时长
// 返回 true: 条件满足; 返回 false: 超时
bool ok = cv.wait_for(lock, 5s, [&] { return !queue.empty(); });

if (ok) {
    // 条件满足
} else {
    // 超时
}
```

notify_one vs notify_all

```
cv.notify_one();    // 唤醒一个等待的线程
cv.notify_all();    // 唤醒所有等待的线程
```

- notify_one: 每次只有一个线程能处理（如队列放入一个元素）
- notify_all: 关闭/停止时，需要唤醒所有线程让它们退出

常见陷阱

问题	原因	解决方案
虚假唤醒导致错误	wait 没有 predicate	始终使用带 predicate 的 wait
通知丢失	先 notify 后 wait	修改条件和通知都在锁内，等待在锁内检查
死锁	wait 时持有其他锁	wait 前释放所有其他锁
用 lock_guard 配合条件变量	lock_guard 不支持临时解锁	改用 unique_lock

思考题

1. 为什么 `condition_variable::wait` 必须配合 `unique_lock` 而不能用 `lock_guard`?
2. 如果生产者在消费者调用 `wait` 之前就调用了 `notify_one`, 会发生什么?
3. `notify_one` 和 `notify_all` 在性能上有什么区别? 什么时候应该用哪个?

std::shared_mutex + std::shared_lock — 读写锁

概念

读多写少的场景中, 普通 `mutex` 让读者也互相阻塞, 浪费并发性。

`std::shared_mutex` 允许: - **多个读者**同时持有锁 (共享读锁) - **写者独占锁**, 写时没有其他读者或写者

基本用法

```
#include <shared_mutex>
#include <unordered_map>

std::shared_mutex rw_mutex;
std::unordered_map<int, std::string> config;

// 读操作: 共享锁, 多线程可同时持有
std::string get_config(int key) {
    std::shared_lock lock(rw_mutex); // 共享读锁
    return config.at(key);
}

// 写操作: 独占锁, 同一时刻只有一个线程
void set_config(int key, const std::string& val) {
    std::unique_lock lock(rw_mutex); // 独占写锁
    config[key] = val;
}
```

锁的兼容性

当前持有	新请求共享锁	新请求独占锁
无锁	允许	允许
共享锁 (一个或多个)	允许	阻塞
独占锁	阻塞	阻塞

shared_lock vs unique_lock vs lock_guard

```
std::shared_mutex mtx;
```

```
// shared_lock: 共享读锁
std::shared_lock lock(mtx);    // 读操作用这个

// unique_lock: 独占写锁
std::unique_lock lock(mtx);    // 写操作用这个

// lock_guard: 对 shared_mutex 加的是独占锁!
std::lock_guard lock(mtx);     // 等价于独占锁, 读操作不要用这个
```

注意：读操作必须用 `std::shared_lock`，否则失去读写锁的并发优势。

性能对比

普通 `mutex`（8个读线程，每次读耗时 10ms）：

```
线程1 读 ██████████
线程2 读          ██████████
线程3 读                ██████████
总耗时 ≈ 80ms（串行）
```

`shared_mutex`（8个读线程）：

```
线程1 读 ██████████
线程2 读 ██████████
线程3 读 ██████████
总耗时 ≈ 10ms（并行）
```

mutable shared_mutex

`const` 成员函数也需要加锁，用 `mutable` 修饰：

```
class ConfigStore {
public:
    std::string get(int key) const {
        std::shared_lock lock(mutex_);    // const 函数中加共享锁
        auto it = data_.find(key);
        return it != data_.end() ? it->second : "";
    }

    void set(int key, std::string val) {
        std::unique_lock lock(mutex_);    // 独占写锁
        data_[key] = std::move(val);
    }

private:
    mutable std::shared_mutex mutex_;
    std::unordered_map<int, std::string> data_;
};
```

升级锁（注意事项）

`shared_mutex` 不支持从共享锁升级为独占锁（会死锁）：

```
std::shared_lock slock(rw_mutex);
// 错误：持有共享锁时再获取独占锁 → 死锁！
std::unique_lock ulock(rw_mutex);

// 正确：先释放共享锁，再获取独占锁
slock.unlock();
std::unique_lock ulock(rw_mutex);
```

常见陷阱

问题	原因	解决方案
读操作用了独占锁	用 <code>lock_guard</code> 或 <code>unique_lock</code> 读	读操作改用 <code>shared_lock</code>
写操作用了共享锁	用 <code>shared_lock</code> 写	写操作改用 <code>unique_lock</code>
持共享锁时升级为独占锁	死锁	先释放共享锁，再获取独占锁
忘记 <code>mutable</code>	<code>const</code> 函数无法加锁	<code>mutex</code> 声明为 <code>mutable</code>

思考题

1. 什么场景下读写锁比普通 `mutex` 性能更好？什么场景下反而更差？
2. 为什么 `shared_mutex` 不支持从共享锁升级为独占锁？
3. 如果写操作非常频繁，读写锁还有优势吗？

`std::atomic` — 无锁原子操作

概念

对于简单的计数器和标志位，用 `mutex` 加锁开销太大。`std::atomic` 利用 CPU 硬件指令保证操作的原子性，**无需加锁**。

“原子”意味着操作不可分割——其他线程要么看到操作前的状态，要么看到操作后的状态，不会看到中间状态。

基本用法

```
#include <atomic>

std::atomic<uint64_t> request_count{0};
std::atomic<bool> shutdown{false};
std::atomic<int*> latest_ptr{nullptr};
```

常用操作

```
std::atomic<int> val{10};

// 读取
int current = val.load();
int current2 = val; // 隐式转换, 等价于 load()

// 写入
val.store(42);
val = 42; // 等价于 store()

// 原子加法 (返回旧值)
int old = val.fetch_add(5); // old == 42, val == 47
val += 5; // 等价写法, 不返回旧值

// 原子减法
val.fetch_sub(3);

// 原子位运算
val.fetch_and(0xFF);
val.fetch_or(0x01);
```

exchange — 原子读-改-写

exchange 原子地 “读取旧值并写入新值”，常用于实现只执行一次的逻辑：

```
std::atomic<bool> initialized{false};

void init_once() {
    // 错误：读和写是两步，存在竞争
    if (!initialized) {
        initialized = true;
        // 两个线程可能同时读到 false，都进入这里！
        do_init();
    }

    // 正确：exchange 是原子的读-改-写
    if (initialized.exchange(true)) return; // 已初始化，直接返回
    // 只有第一个调用的线程会看到 false，其余线程看到 true 直接返回
    do_init();
}
```

compare_exchange — CAS（比较并交换）

CAS 是无锁算法的基础：

```
std::atomic<int> val{0};

int expected = 0;
int desired = 1;

// 若 val == expected, 则将 val 设为 desired, 返回 true
// 若 val != expected, 则将 expected 更新为 val 的当前值, 返回 false
bool success = val.compare_exchange_strong(expected, desired);
```

无锁自增（CAS 循环）：

```
void atomic_increment(std::atomic<int>& val) {
    int expected = val.load();
    while (!val.compare_exchange_weak(expected, expected + 1)) {
        // expected 已被更新为当前值, 重试
    }
}
```

atomic vs mutex 对比

特性	std::atomic	std::mutex
适用范围	单个简单变量	复杂数据结构
性能	极高（硬件指令）	较高（系统调用）
使用难度	简单	简单
多变量一致性	不支持	支持

内存顺序（简介）

std::atomic 操作默认使用 memory_order_seq_cst（顺序一致性），最安全。对于简单计数器，可以用更宽松的顺序提升性能：

```
// 顺序一致（默认，最安全）
counter.fetch_add(1);

// 宽松顺序（只保证原子性，适用于只关心最终值的计数器）
counter.fetch_add(1, std::memory_order_relaxed);
```

建议：初学者始终使用默认顺序一致性。

常见陷阱

问题	原因	解决方案
非原子的复合操作	if (a) { a = false; } 不是原子的	用 exchange 或 compare_exchange
多变量一致性	两个 atomic 无法原子地同时更新	用 mutex 保护多变量操作
误以为 atomic 万能	atomic 只保证单个变量的原子性	复杂逻辑仍需 mutex

思考题

1. ++counter 和 counter.fetch_add(1) 有什么区别?
2. 什么情况下应该用 atomic<bool> 而不是 mutex + bool?
3. 为什么 compare_exchange_weak 可能虚假失败? 什么时候用 weak, 什么时候用 strong?

std::chrono — 时间处理

概念

std::chrono 是 C++11 引入的类型安全时间库, 解决了传统 C 时间函数 (time_t、sleep()) 类型不安全、精度不足的问题。

时间字面量

```
#include <chrono>
using namespace std::chrono_literals;

auto t1 = 1s;      // 1 秒
auto t2 = 500ms;   // 500 毫秒
auto t3 = 100us;   // 100 微秒
auto t4 = 30min;   // 30 分钟
auto t5 = 2h;      // 2 小时

// 时间段运算
auto total = 1h + 30min + 45s;
```

时间点与时钟

```
// steady_clock: 单调递增, 不受系统时间调整影响, 适合计时
auto now = std::chrono::steady_clock::now();

// system_clock: 系统时钟, 可以转换为日历时间
auto sys_now = std::chrono::system_clock::now();

// 计算时间差
auto start = std::chrono::steady_clock::now();
// ... 执行操作 ...
auto end = std::chrono::steady_clock::now();
auto elapsed = end - start;

// 转换为毫秒数
double ms = std::chrono::duration<double, std::milli>(elapsed).count();
std::cout << " 耗时: " << ms << " ms\n";
```

时间转换

```
std::chrono::seconds sec(90);

// 整数截断
auto min = std::chrono::duration_cast<std::chrono::minutes>(sec);
std::cout << min.count() << "\n"; // 1 (截断)

// 浮点保留精度
double secs = std::chrono::duration<double>(1500ms).count();
std::cout << secs << "\n"; // 1.5
```

定时等待 vs 可中断等待

```
// sleep_for: 固定延迟, 无法提前唤醒
std::this_thread::sleep_for(30s); // 必须等满 30 秒

// wait_for: 可以被 notify 提前唤醒 (配合 condition_variable)
std::unique_lock lock(mutex_);
cv_.wait_for(lock, 30s, [this] { return stop_flag_.load(); });
// 等待 30s, 或被 stop() 提前唤醒
```

后台服务的可中断等待模式:

```
class BackgroundService {
    std::mutex mutex_;
    std::condition_variable cv_;
    std::atomic<bool> stop_flag_{false};
    std::thread thread_;

    void run() {
        while (true) {
            std::unique_lock lock(mutex_);
            // 等待 interval, 或被 stop() 提前唤醒
            cv_.wait_for(lock, interval_, [this] { return stop_flag_.load(); });

            if (stop_flag_.load()) break;

            lock.unlock();
            do_periodic_work(); // 定期执行的任务
        }
    }

public:
    void stop() {
        stop_flag_ = true;
        cv_.notify_all(); // 立即唤醒, 不需要等到下次
    }
};
```

```

        if (thread_.joinable()) thread_.join();
    }
};

```

	sleep_for	wait_for
能否提前唤醒	不能	能（通过 notify）
需要 mutex	不需要	需要
适用场景	固定延迟	可中断的定时等待

TTL（存活时间）过期机制

```

struct CacheEntry {
    std::string value;
    std::chrono::steady_clock::time_point expire_at{
        std::chrono::steady_clock::time_point::max() // 默认永不过期
    };

    bool is_expired() const {
        return expire_at != std::chrono::steady_clock::time_point::max()
            && std::chrono::steady_clock::now() > expire_at;
    }
};

// 写入时设置过期时间
void put(const std::string& key, std::string value, std::chrono::seconds ttl) {
    auto expire_at = (ttl.count() > 0)
        ? std::chrono::steady_clock::now() + ttl
        : std::chrono::steady_clock::time_point::max();

    cache_[key] = CacheEntry{std::move(value), expire_at};
}

```

常见陷阱

问题	原因	解决方案
用 sleep_for 实现可中断等待	sleep_for 无法被提前唤醒	改用 cv.wait_for
时间点比较用 system_clock	系统时间可能被调整	用 steady_clock
duration_cast 精度丢失	整数截断	用浮点 duration<double>
忘记 using namespace std::chrono_literals	1s、500ms 等字面量无法使用	添加 using 声明

思考题

1. steady_clock 和 system_clock 各适合什么场景？
2. 为什么后台服务的定时等待应该用 wait_for 而不是 sleep_for？
3. 如何实现一个精确的限速器（rate limiter），保证每秒最多处理 N 个请求？

std::variant + std::visit — 类型安全的多态值

概念

当一个变量可能存储多种不同类型的值时，传统做法是用继承体系。std::variant 提供了更简洁、性能更好的替代方案。

std::variant<T1, T2, ...> 是类型安全的联合体：- 编译期确定可能的类型集合 - 同一时刻只存储其中一种类型的值 - 访问时进行类型检查

基本用法

```
#include <variant>
#include <string>

// 可以存储 int、double 或 string 的 variant
using ConfigValue = std::variant<int, double, std::string>;

ConfigValue port      = 8080;
ConfigValue timeout   = 30.5;
ConfigValue host       = std::string("localhost");

// 查询当前存储的类型索引 (0=int, 1=double, 2=string)
std::cout << port.index(); // 0
std::cout << host.index(); // 2
```

取值方式

std::get — 按类型取值

```
ConfigValue val = 8080;

int port = std::get<int>(val); // 8080

// 类型不匹配时抛出 std::bad_variant_access
try {
    double d = std::get<double>(val); // 抛出异常!
} catch (const std::bad_variant_access& e) {
    std::cout << " 类型不匹配\n";
}
```

std::get_if — 安全取值（不抛异常）

```
ConfigValue val = std::string("localhost");

if (auto* p = std::get_if<std::string>(&val)) {
    std::cout << " 字符串值: " << *p << "\n";
}
```

```

} else {
    std::cout << " 不是字符串\n";
}

```

std::holds_alternative — 检查类型

```

if (std::holds_alternative<int>(val)) {
    std::cout << " 是整数: " << std::get<int>(val) << "\n";
}

```

std::visit — 对所有可能类型执行操作

```

ConfigValue val = 8080;

```

// 泛型 lambda (推荐)

```

std::visit([](auto&& v) {
    using T = std::decay_t<decltype(v)>;
    if constexpr (std::is_same_v<T, int>)
        std::cout << " 整数: " << v << "\n";
    else if constexpr (std::is_same_v<T, double>)
        std::cout << " 浮点: " << v << "\n";
    else if constexpr (std::is_same_v<T, std::string>)
        std::cout << " 字符串: " << v << "\n";
}, val);

```

// 重载访问者 (overloaded pattern)

```

struct Printer {
    void operator()(int v) { std::cout << "int: " << v << "\n"; }
    void operator()(double v) { std::cout << "double: " << v << "\n"; }
    void operator()(const std::string& v) { std::cout << "string: " << v << "\n"; }
};

```

```

std::visit(Printer{}, val);

```

// visit 有返回值

```

std::string str = std::visit([](auto&& v) -> std::string {
    using T = std::decay_t<decltype(v)>;
    if constexpr (std::is_same_v<T, int>)
        return "int:" + std::to_string(v);
    else if constexpr (std::is_same_v<T, double>)
        return "double:" + std::to_string(v);
    else
        return "string:" + v;
}, val);

```

与继承体系的对比

特性	继承体系	std::variant
类型集合	开放（可随时添加子类）	封闭（编译期固定）
内存分配	堆分配	栈上存储
虚函数开销	有	无
类型安全	运行时（dynamic_cast）	编译期 + 运行时
适用场景	类型集合不固定	类型集合固定

常见陷阱

问题	原因	解决方案
std::get<T> 抛异常	类型不匹配	先用 holds_alternative 检查，或用 get_if
visit 漏掉某个类型	if constexpr 不完整	确保覆盖所有类型，或加 static_assert
整数字面量类型错误	42 是 int，不是 int64_t	用 int64_t(42) 或 42LL

思考题

1. std::variant 和 union 有什么区别？
2. 什么时候应该用 std::variant，什么时候应该用继承体系？
3. 如何用 std::visit 实现一个通用的序列化函数，将 variant 转换为 JSON 字符串？

std::any — 类型擦除的通用容器

概念

std::any 可以存储任意可拷贝类型的值，类型在运行时确定。适合需要存储“不知道是什么类型”的场景，如插件系统的配置项、事件的附加数据等。

与 std::variant 的区别：- variant：类型集合在编译期固定 - any：类型集合完全开放，运行时确定

基本用法

```
#include <any>

std::any data;

data = std::string("hello"); // 存储 string
data = 42;                  // 覆盖为 int
data = 3.14;                // 覆盖为 double

// 检查是否有值
if (data.has_value()) {
    std::cout << " 有值\n";
}
```

```
// 检查类型
if (data.type() == typeid(double)) {
    std::cout << " 当前是 double\n";
}

// 重置为空
data.reset();
```

取值方式

`std::any_cast` — 直接取值（可能抛异常）

```
std::any a = std::string("world");

std::string s = std::any_cast<std::string>(a); // "world"

// 类型不匹配时抛出 std::bad_any_cast
try {
    int n = std::any_cast<int>(a); // 抛出异常!
} catch (const std::bad_any_cast& e) {
    std::cout << " 类型不匹配\n";
}
```

`std::any_cast<T*>` — 安全取值（不抛异常）

```
std::any a = 42;

// 传入指针，类型匹配时返回指向值的指针，不匹配时返回 nullptr
if (auto* p = std::any_cast<int>(&a)) {
    std::cout << " 整数值: " << *p << "\n";
} else {
    std::cout << " 不是整数\n";
}
```

实际应用：事件系统

```
struct Event {
    std::string type;
    std::any payload; // 不同事件携带不同类型的数据
};

// 鼠标点击事件
struct MouseEvent { int x, y; };
Event click{"mouse_click", MouseEvent{100, 200}};
```

```

// 键盘事件
struct KeyEvent { int keycode; bool shift; };
Event key{"key_press", KeyEvent{65, true}};

// 处理事件
void handle(const Event& e) {
    if (e.type == "mouse_click") {
        if (auto* me = std::any_cast<MouseEvent>(&e.payload)) {
            std::cout << " 点击位置: " << me->x << ", " << me->y << "\n";
        }
    } else if (e.type == "key_press") {
        if (auto* ke = std::any_cast<KeyEvent>(&e.payload)) {
            std::cout << " 按键: " << ke->keycode << "\n";
        }
    }
}

```

模板包装：类型安全的 set/get

```

class Context {
    std::any data_;

public:
    template<typename T>
    void set(T value) {
        data_ = std::any(std::move(value));
    }

    template<typename T>
    std::optional<T> get() const {
        auto* p = std::any_cast<T>(&data_); // 指针版本，不抛异常
        if (p) return *p;
        return std::nullopt;
    }
};

// 使用
Context ctx;
ctx.set(std::string("user_123"));
auto id = ctx.get<std::string>(); // optional<string>{"user_123"}
auto num = ctx.get<int>();       // nullopt (类型不匹配)

```

variant vs any 对比

特性	<code>std::variant</code>	<code>std::any</code>
类型集合	编译期固定	完全开放
类型安全	编译期检查	运行时检查
内存分配	栈上（固定大小）	可能堆分配（SOO 优化）
适用场景	已知类型集合	未知类型集合

常见陷阱

问题	原因	解决方案
<code>any_cast<T></code> 抛异常	类型不匹配	用 <code>any_cast<T*></code> 指针版本，检查 <code>nullptr</code>
存储不可拷贝类型	<code>any</code> 要求类型可拷贝	用 <code>shared_ptr</code> 包装不可拷贝类型
误用引用版 <code>any_cast</code>	<code>any_cast<T&>(a)</code> 类型不匹配时抛异常	用指针版本 <code>any_cast<T>(&a)</code>

思考题

1. `std::any` 和 `void*` 有什么本质区别？
2. 什么时候应该用 `std::any`，什么时候应该用 `std::variant`？
3. 如何用 `std::any` 实现一个类型安全的属性系统（property system）？

`std::future` + `std::promise` + `std::packaged_task` — 异步结果传递

概念

线程之间需要传递计算结果。C++ 提供了一套“异步通信”机制：

- `std::promise`：写端，在某个线程中设置结果
- `std::future`：读端，在另一个线程中等待并获取结果
- `std::packaged_task`：将可调用对象包装为可获取返回值的任务

`std::promise` + `std::future`

```
#include <future>
#include <thread>

std::promise<int> promise;
std::future<int> future = promise.get_future();

// 在另一个线程中设置结果
std::thread t([p = std::move(promise)]() mutable {
    int result = compute_something();
    p.set_value(result); // 设置结果，唤醒等待的 future
});

// 主线程等待结果（阻塞直到 set_value 被调用）
```

```
int result = future.get();
t.join();
```

传递异常:

```
std::promise<int> promise;
auto future = promise.get_future();

std::thread t([p = std::move(promise)]() mutable {
    try {
        throw std::runtime_error(" 计算失败");
    } catch (...) {
        p.set_exception(std::current_exception());
    }
});
t.join();

try {
    future.get(); // 重新抛出线程内的异常
} catch (const std::exception& e) {
    std::cout << " 捕获到: " << e.what() << "\n";
}
```

std::packaged_task

将可调用对象包装为任务, 自动管理 promise/future 对:

```
std::packaged_task<int(int, int)> task([](int a, int b) {
    return a + b;
});

std::future<int> future = task.get_future(); // 先拿 future

std::thread t(std::move(task), 3, 4); // packaged_task 只能移动!
t.join();

int result = future.get(); // result == 7
```

为什么 packaged_task 不能放入 std::function?

std::function 要求内部对象可拷贝, 但 packaged_task 含有 promise, 只能移动。

```
// 错误: packaged_task 不可拷贝
std::function<void()> f = std::move(task); // 编译错误!

// 正确: 用 shared_ptr 包装, shared_ptr 可以拷贝
auto task_ptr = std::make_shared<std::packaged_task<int()>>(
    [] { return 42; }
```

```
);
auto future = task_ptr->get_future();

std::function<void()> f = [task_ptr] { (*task_ptr)(); }; // OK
f();
int result = future.get(); // 42
```

std::async — 最简便的异步方式

```
// std::async 自动创建线程并返回 future
auto future = std::async(std::launch::async, [] {
    return compute_something();
});

// 做其他事情...

auto result = future.get(); // 需要时再等待
```

future 的状态

```
std::future<int> f;
f.valid(); // false: 默认构造, 无效

std::promise<int> p;
f = p.get_future();
f.valid(); // true: 关联了 promise

p.set_value(42);
int val = f.get(); // 获取结果
f.valid(); // false: get() 只能调用一次!
```

std::shared_future — 多线程共享结果

```
std::promise<int> p;
std::shared_future<int> sf = p.get_future().share();

// 多个线程可以同时等待同一个结果
std::thread t1([sf] { std::cout << sf.get() << "\n"; });
std::thread t2([sf] { std::cout << sf.get() << "\n"; });

p.set_value(42);
t1.join();
t2.join();
```

常见陷阱

问题	原因	解决方案
get() 调用两次 packaged_task 无法存入 function 忘记在执行前获取 future	future 只能 get 一次 packaged_task 不可拷贝 get_future() 只能调用一次	用 shared_future 或只 get 一次 用 shared_ptr<packaged_task> 包装 创建 task 后立即 get_future()
future 析构时阻塞	std::async 返回的 future 析构时等待完成	保存 future 或用 detach

思考题

1. std::promise 和 std::packaged_task 都能传递结果，有什么区别？各适合什么场景？
2. 为什么 future::get() 只能调用一次？如果需要多次读取结果怎么办？
3. 如何用 promise/future 实现一个简单的线程池？

模板、完美转发与参数包展开

一、函数模板基础

```
// 普通函数: 只接受 int
int add(int a, int b) { return a + b; }

// 函数模板: 接受任意类型
template<typename T>
T add(T a, T b) { return a + b; }

add(1, 2);           // T = int
add(1.5, 2.5);       // T = double
```

二、可变参数模板 (Variadic Templates)

typename... Args 表示零个或多个类型，Args... args 表示对应的参数包：

```
// 接受任意数量、任意类型的参数
template<typename... Args>
void print_all(Args... args) {
    // args 是参数包，不能直接使用，必须"展开"
}
```

sizeof... — 获取参数个数

```
template<typename... Args>
void show_count(Args... args) {
```

```

    std::cout << " 参数个数: " << sizeof...(Args) << "\n";
    std::cout << " 参数个数: " << sizeof...(args) << "\n"; // 等价
}

show_count(1, "hello", 3.14); // 输出: 参数个数: 3

```

三、包展开语法

... 放在模式的右侧，将模式应用到包中的每个元素：

```

template<typename... Args>
void example(Args... args) {
    // 展开为函数调用的参数列表
    some_function(args...); // some_function(a0, a1, a2)

    // 展开为初始化列表
    int arr[] = {args...}; // int arr[] = {a0, a1, a2}

    // 对每个元素应用表达式（模式展开）
    some_function((args + 1)...); // some_function(a0+1, a1+1, a2+1)
    some_function(std::move(args)...); // some_function(move(a0), move(a1), ...)
}

```

递归展开 (C++11 方式)

```

// 递归终止条件
void print() {}

// 递归展开: 每次处理第一个参数, 剩余的递归处理
template<typename T, typename... Rest>
void print(T first, Rest... rest) {
    std::cout << first << " ";
    print(rest...); // 递归, 参数包少一个
}

print(1, "hello", 3.14); // 输出: 1 hello 3.14

```

折叠表达式 (C++17, 推荐)

折叠表达式让包展开更简洁，无需递归：

```

// 一元右折叠: (args op ...)
// 展开为: a0 op (a1 op (a2 op ...))
template<typename... Args>
auto sum(Args... args) {
    return (args + ...); // a0 + (a1 + (a2 + ...))
}

```

```

// 一元左折叠: (... op args)
// 展开为: (... op (a0 op a1)) op a2
template<typename... Args>
auto sum_left(Args... args) {
    return (... + args);
}

// 二元右折叠: (args op ... op init)
// 展开为: a0 op (a1 op (a2 op init))
template<typename... Args>
auto sum_with_init(Args... args) {
    return (args + ... + 0); // 初始值为 0, 空包时返回 0
}

sum(1, 2, 3, 4); // 10

```

常用折叠表达式:

```

// 逻辑与 (所有条件都为 true)
template<typename... Args>
bool all_positive(Args... args) {
    return (... && (args > 0));
}

// 逻辑或 (至少一个条件为 true)
template<typename... Args>
bool any_positive(Args... args) {
    return (... || (args > 0));
}

// 打印所有参数 (逗号运算符折叠)
template<typename... Args>
void print_all(Args... args) {
    ((std::cout << args << " "), ...); // 展开为逗号表达式
    std::cout << "\n";
}

print_all(1, "hello", 3.14); // 输出: 1 hello 3.14

```

四、万能引用与完美转发

万能引用 (Universal Reference)

T&& 在模板中是万能引用, 不是右值引用:

```

template<typename T>
void wrapper(T&& arg) { // T&& 是万能引用
    // 传入左值: T 推导为 T&, T&& 折叠为 T& (左值引用)
}

```

```

    // 传入右值: T 推导为 T, T&& 为 T&& (右值引用)
}

int x = 42;
wrapper(x);    // T = int&, arg 是左值引用
wrapper(42);   // T = int, arg 是右值引用

```

std::forward — 完美转发

在函数体内转发参数时，必须用 `std::forward<T>(t)` 保持原始值类别：

```

template<typename T>
void relay(T&& arg) {
    // 错误: arg 在函数体内是左值 (有名字), 右值信息丢失
    target(arg);

    // 正确: forward 恢复原始值类别
    target(std::forward<T>(arg));
}

```

参数包的完美转发

```

template<typename F, typename... Args>
auto call(F&& f, Args&&... args) {
    // 对参数包中的每个元素应用 forward
    return std::forward<F>(f)(std::forward<Args>(args)...);
    // 展开为: f(forward<A0>(a0), forward<A1>(a1), ...)
}

```

五、lambda 中的参数包捕获 (C++20)

C++20 允许在 lambda 中捕获参数包：

```

template<typename F, typename... Args>
auto defer(F&& f, Args&&... args) {
    // C++20: 包展开捕获, 每个参数独立捕获并完美转发
    return [f = std::forward<F>(f), ...args = std::forward<Args>(args)]() mutable {
        return std::invoke(std::move(f), std::move(args)...);
    };
}

// 使用
auto task = defer([](int a, int b) { return a + b; }, 3, 4);
int result = task(); // 7

```

C++17 的替代方案（用 tuple 存储参数包）：

```

template<typename F, typename... Args>
auto defer_cpp17(F&& f, Args&&... args) {
    auto bound_args = std::make_tuple(std::forward<Args>(args)...);
    return [f = std::forward<F>(f),
            args = std::move(bound_args)]() mutable {
        return std::apply(std::move(f), std::move(args));
    };
}

```

六、std::invoke — 统一调用接口

std::invoke 统一了所有可调用对象的调用方式：

```

#include <functional>

int add(int a, int b) { return a + b; }
std::invoke(add, 1, 2); // → 3

auto mul = [](int a, int b) { return a * b; };
std::invoke(mul, 3, 4); // → 12

struct Calculator {
    int base{10};
    int add(int x) const { return base + x; }
};

Calculator calc;
std::invoke(&Calculator::add, calc, 5); // → 15 (传对象)
std::invoke(&Calculator::add, &calc, 5); // → 15 (传指针)
std::invoke(&Calculator::base, calc); // → 10 (成员变量指针)

```

为什么需要 std::invoke?

```

auto f = &Calculator::add;
// f(calc, 5) ← 编译错误! 成员函数指针不能直接调用
// (calc.*f)(5) ← 正确, 但语法复杂

std::invoke(f, calc, 5); // 统一语法

```

std::invoke_result_t — 编译期推导返回类型

```

using R = std::invoke_result_t<F, Args...>;

// 示例
using R1 = std::invoke_result_t<decltype(add), int, int>; // int
using R2 = std::invoke_result_t<decltype(&Calculator::add), Calculator, int>; // int

```

七、综合示例：通用日志函数

```
#include <iostream>
#include <functional>

// 类型安全的格式化打印（类似 printf，但类型安全）
template<typename... Args>
void log(const std::string& level, Args&&... args) {
    std::cout << "[" << level << "]" ";
    // 折叠表达式：依次打印所有参数
    ((std::cout << std::forward<Args>(args) << " "), ...);
    std::cout << "\n";
}

log("INFO", " 用户", 42, " 登录成功");
// 输出：[INFO] 用户 42 登录成功

// 通用工厂函数：完美转发构造参数
template<typename T, typename... Args>
std::unique_ptr<T> make(Args&&... args) {
    return std::make_unique<T>(std::forward<Args>(args)...);
}
```

常见陷阱

问题	原因	解决方案
成员函数指针调用失败	直接用 <code>f(args...)</code> 不支持成员函数指针	改用 <code>std::invoke(f, args...)</code>
移动语义失效	忘记 <code>std::forward</code> ，右值退化为左值	始终用 <code>std::forward<T>(t)</code> 转发
空参数包折叠表达式	<code>(args + ...)</code> 空包时未定义	用二元折叠 <code>(args + ... + 0)</code> 提供初始值
C++17 不支持包展开捕获	<code>[...args = ...]</code> 是 C++20 特性	用 <code>std::tuple + std::apply</code> 替代

思考题

1. `std::forward<T>(t)` 和 `std::move(t)` 有什么区别？
2. 折叠表达式 `(args + ...)` 和 `(... + args)` 在结果上有什么区别？什么时候有影响？
3. 如何用可变参数模板实现一个 `zip` 函数，将多个 `vector` 合并为 `vector of tuple`？
4. 为什么 `[...args = std::forward<Args>(args)]` 需要 `mutable` 修饰 `lambda`？

哈希分片（Sharding）— 降低锁竞争

概念

当并发量很高时，即使是读写锁也会成为瓶颈——所有写操作仍然互相阻塞。

分片（Sharding）将数据分散到多个独立的桶（shard）中，每个桶有自己的锁。不同桶的操作完全独立，可以真正并行，将锁竞争降低为原来的 $1/N$ 。

核心思路

不分片（1把锁）：

写A 
写B  ← 必须等 A 完成
写C  ← 必须等 B 完成

分片（3把锁，A/B/C 在不同 shard）：

写A 
写B  ← 与 A 并行！
写C  ← 与 A/B 并行！

哈希路由

将 key 映射到 shard 的标准方式：

```
size_t shard_index = std::hash<std::string>{}(key) % num_shards;
```

性质：- 同一个 key 始终映射到同一个 shard（一致性）- 不同 key 均匀分布到各个 shard（均匀性）- 路由计算 $O(1)$

完整实现示例

```
#include <vector>
#include <shared_mutex>
#include <unordered_map>
#include <memory>

template<typename V>
class ShardedMap {
    struct Shard {
        mutable std::shared_mutex mutex;
        std::unordered_map<std::string, V> data;
    };

    std::vector<std::unique_ptr<Shard>> shards_;

    Shard& get_shard(const std::string& key) {
        size_t idx = std::hash<std::string>{}(key) % shards_.size();
        return *shards_[idx];
    }

public:
    explicit ShardedMap(size_t num_shards = 16) {
        for (size_t i = 0; i < num_shards; ++i)
            shards_.push_back(std::make_unique<Shard>());
    }
};
```

```

std::optional<V> get(const std::string& key) const {
    auto& shard = const_cast<ShardedMap*>(this)->get_shard(key);
    std::shared_lock lock(shard.mutex); // 只锁这一个 shard
    auto it = shard.data.find(key);
    if (it == shard.data.end()) return std::nullopt;
    return it->second;
}

void put(const std::string& key, V value) {
    auto& shard = get_shard(key);
    std::unique_lock lock(shard.mutex); // 只锁这一个 shard
    shard.data[key] = std::move(value);
}
};

```

分片数量的选择

```

// 分片数越多，锁竞争越低，但内存开销越大
// 通常选择 2 的幂次，方便用位运算替代取模

// 取模（通用）
size_t idx = hash % num_shards;

// 位运算（num_shards 必须是 2 的幂次，性能更好）
size_t idx = hash & (num_shards - 1);

```

经验法则： - 读多写少：16~64 个 shard 通常足够 - 写密集：可以增加 CPU 核心数的 2~4 倍 - 分片数不是越多越好，过多会增加内存和管理开销

跨分片操作与死锁

当一个操作涉及多个 key（可能在不同 shard）时，必须注意死锁：

```

// 错误：不同线程可能以不同顺序加锁，导致死锁
void swap_bad(const std::string& key1, const std::string& key2) {
    auto& shard1 = get_shard(key1);
    auto& shard2 = get_shard(key2);
    std::unique_lock lock1(shard1.mutex); // 线程 A 先锁 shard0
    std::unique_lock lock2(shard2.mutex); // 线程 A 等 shard1
    // 线程 B 先锁 shard1，等 shard0 → 死锁!
}

// 正确方式 1：按 shard 索引排序加锁
void swap_safe(const std::string& key1, const std::string& key2) {
    size_t idx1 = std::hash<std::string>{}(key1) % shards_.size();
    size_t idx2 = std::hash<std::string>{}(key2) % shards_.size();
}

```

```

    if (idx1 > idx2) std::swap(idx1, idx2); // 确保 idx1 <= idx2
    std::unique_lock lock1(shards_[idx1]->mutex);
    std::unique_lock lock2(shards_[idx2]->mutex);
    // 安全地操作...
}

// 正确方式 2: std::lock 同时获取多把锁
void swap_std_lock(const std::string& key1, const std::string& key2) {
    auto& shard1 = get_shard(key1);
    auto& shard2 = get_shard(key2);
    std::unique_lock lock1(shard1.mutex, std::defer_lock);
    std::unique_lock lock2(shard2.mutex, std::defer_lock);
    std::lock(lock1, lock2); // 原子地获取两把锁，内部避免死锁
}

```

常见陷阱

问题	原因	解决方案
跨 shard 操作死锁	不同线程以不同顺序加锁	按 shard 索引排序加锁
分片不均匀	哈希函数质量差	使用标准库 <code>std::hash</code>
全局统计不准确	各 shard 的计数需要汇总	用 <code>std::atomic</code> 计数器，或遍历所有 shard
分片数为 0	取模除以 0	构造时断言 <code>shard_count > 0</code>

思考题

1. 分片数量对性能的影响是什么？如何选择合适的分片数？
2. 如果需要遍历所有数据（如统计总数），分片设计会带来什么挑战？
3. 一致性哈希（Consistent Hashing）和普通哈希分片有什么区别？各适合什么场景？

事务与死锁避免

一、死锁

什么是死锁？

死锁发生在多个线程互相等待对方持有的资源，导致所有线程永远无法继续：

线程A：持有资源1 → 等待资源2

线程B：持有资源2 → 等待资源1

→ 两人互相等待，永远无法继续 → 死锁！

死锁的四个必要条件（Coffman 条件）

1. **互斥**：资源同一时刻只能被一个线程持有
2. **持有并等待**：线程持有资源的同时等待其他资源
3. **不可抢占**：资源只能由持有者主动释放

4. 循环等待：存在线程等待链形成环路

破坏任意一个条件即可避免死锁。最实用的方法是破坏“循环等待”：所有线程按相同顺序获取锁。

按序加锁

```
// 错误：不同线程加锁顺序不同，可能死锁
void thread_A() { lock(mutex_a); lock(mutex_b); /* ... */ }
void thread_B() { lock(mutex_b); lock(mutex_a); /* ... */ }

// 正确：所有线程按固定顺序加锁（破坏循环等待）
void safe_operation(std::mutex& m1, std::mutex& m2) {
    // 按地址排序，确保所有线程顺序一致
    auto* first = (&m1 < &m2) ? &m1 : &m2;
    auto* second = (&m1 < &m2) ? &m2 : &m1;

    std::lock_guard lg1(*first);
    std::lock_guard lg2(*second);
    // 不可能出现循环等待
}
```

std::lock — 同时获取多把锁

std::lock(m1, m2, ...) 原子地获取多把锁，内部使用尝试-回退（try-and-backoff）算法避免死锁：

```
std::mutex m1, m2;

void safe_swap() {
    std::unique_lock lock1(m1, std::defer_lock); // 延迟加锁
    std::unique_lock lock2(m2, std::defer_lock);
    std::lock(lock1, lock2); // 原子地获取两把锁，内部自动避免死锁
    // 安全地操作...
}

// C++17: scoped_lock 更简洁
void safe_swap_cpp17() {
    std::scoped_lock lock(m1, m2); // 等价于上面的写法
    // 安全地操作...
}
```

二、事务

事务的 ACID 特性

特性	含义	示例
A (原子性)	操作要么全部成功, 要么全部不发生	转账: 扣款和入账要么都成功, 要么都不发生
C (一致性)	操作前后数据满足约束	转账前后总金额不变
I (隔离性)	事务内能读到自己的未提交写入	事务内修改后立即可读
D (持久性)	提交后数据不丢失	写入磁盘, 重启后仍存在

Write-Set 模式

事务内的所有写操作先记录在本地 write-set 中, commit 时才真正写入。

Read-Your-Writes (读自己的写)

事务内应该能读到自己的未提交写入:

```
Transaction tx;
tx.put("name", "Alice");
auto val = tx.get("name", storage); // 返回 "Alice", 即使还没 commit
```

RAII 自动回滚

```
class Transaction {
    bool committed_{false};

public:
    ~Transaction() {
        if (!committed_) {
            rollback(); // 析构时自动回滚
        }
    }

    void rollback() {
        write_set_.clear();
        committed_ = false;
    }
};

// 使用示例
{
    Transaction tx;
    tx.put("balance", "100");
    // 忘记调用 commit(), 析构时自动 rollback
} // tx 析构, 自动回滚, 数据不会被修改
```

全局大锁实现

最简单的原子提交方式: 整个事务生命周期持有全局锁。

优点：实现简单，绝对安全 缺点：所有事务串行，无并发性

完整使用示例：银行转账

```
TransactionManager mgr;

bool transfer(const std::string& from, const std::string& to, int64_t amount) {
    auto tx = mgr.begin();

    auto from_val = tx->get(from);
    auto to_val = tx->get(to);
    if (!from_val || !to_val) return false;

    int64_t from_balance = std::stoll(*from_val);
    int64_t to_balance = std::stoll(*to_val);
    if (from_balance < amount) return false; // 余额不足

    tx->put(from, std::to_string(from_balance - amount));
    tx->put(to, std::to_string(to_balance + amount));

    return tx->commit(); // 原子提交
}
```

常见陷阱

问题	原因	解决方案
死锁	不同线程以不同顺序加锁	按固定顺序加锁，或用 <code>std::lock</code>
Lost Update（数据丢失）	读在锁外，两个事务读到同一旧值	整个事务生命周期持锁
忘记 rollback	异常导致 commit 未执行	析构函数自动 rollback（RAII）
adopt_lock 用错	锁未被锁定时用 adopt_lock	确保 <code>std::lock</code> 之后再 use adopt_lock

思考题

1. 为什么“读在锁外”会导致 Lost Update？请用具体的时序图说明。
2. 全局大锁保证了原子性，但牺牲了并发性。如何在保证正确性的前提下提高并发性？
3. 数据库的 MVCC（多版本并发控制）是如何避免 Lost Update 的？